Practical No. 7 :-

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint). The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. *(Sample data for reference is given below).*

Process	Allocation				MAX			
Printer	ROM	Hard Disk	RAM	Printer	ROM	Hard Disk	RAM	
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	0	1	2
Excel	1	0	0	0	1	7	5	0
PowerPoint	1	3	5	4	2	3	5	6

Available Resources			
Printer	ROM	Hard Disk	RAM
1	6	2	0

Write a program to :

● Calculate current need of resources by each process.

● Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

```
GNU nano 8.7                                banker.c
#include <stdio.h>

#define P 5   // Number of processes
#define R 4   // Number of resources

int main() {

    int allocation[P][R] = {
        {0, 0, 1, 4},
        {0, 6, 3, 2},
        {0, 0, 1, 2},
        {1, 0, 0, 0},
        {1, 3, 5, 4}
    };

    int max[P][R] = {
        {0, 6, 5, 6},
        {0, 6, 5, 2},
        {0, 0, 1, 2},
        {1, 7, 5, 0},
        {2, 3, 5, 6}
    };

    int available[R] = {1, 6, 2, 0};

    int need[P][R];
    int finish[P] = {0};
    int safeSequence[P];
    int work[R];

    // Calculate NEED matrix
    for(int i = 0; i < P; i++) {
        for(int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Copy available to work
    for(int i = 0; i < R; i++)
        work[i] = available[i];

    int count = 0;

    while(count < P) {
        int found = 0;

        for(int i = 0; i < P; i++) {

            if(finish[i] == 0) {
                int possible = 1;

                for(int j = 0; j < R; j++) {
                    if(need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }
^G Help      ^O Write Out  ^F Where Is   ^K Cut      ^T Execute    ^C Location   M-U Undo     M-A Set Mark  M-] To Bracket  M-B Previous  ^← Back       ^Space Prev Word
^X Exit      ^R Read File  ^\ Replace    ^U Paste    ^J Justify    ^/ Go To Line M-E Redo     M-6 Copy      ^B Where Was    M-F Next      ^→ Forward    ^/ Next Word
```

```
GNU nano 8.7                                banker.c
    int work[R];

    // Calculate NEED matrix
    for(int i = 0; i < P; i++) {
        for(int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Copy available to work
    for(int i = 0; i < R; i++)
        work[i] = available[i];

    int count = 0;

    while(count < P) {
        int found = 0;

        for(int i = 0; i < P; i++) {

            if(finish[i] == 0) {
                int possible = 1;

                for(int j = 0; j < R; j++) {
                    if(need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }

                if(possible) {
                    for(int j = 0; j < R; j++)
                        work[j] += allocation[i][j];

                    safeSequence[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("\nSystem is NOT in safe state.\n");
            return 0;
        }
    }

    printf("\nSystem is in SAFE state.\nSafe Sequence: ");
    for(int i = 0; i < P; i++) {
        printf("P%d ", safeSequence[i]);
    }

    printf("\n");
    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut      ^T Execute    ^C Location   M-U Undo     M-A Set Mark  M-] To Bracket  M-B Previous  ^← Back       ^Space Prev Word
^X Exit      ^R Read File  ^\ Replace    ^U Paste    ^J Justify    ^/ Go To Line M-E Redo     M-6 Copy      ^B Where Was    M-F Next      ^→ Forward    ^/ Next Word
```

```
Barkha@DESKTOP-35GK287 MSYS ~
$ nano banker.c

Barkha@DESKTOP-35GK287 MSYS ~
$ gcc banker.c -o banker
./banker

System is in SAFE state.
Safe Sequence: P1 P2 P3 P4 P0

Barkha@DESKTOP-35GK287 MSYS ~
$ |
```